



FACULTAD
DE INGENIERÍA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Esquemático y Software

“Robotin Matero”

Grupo 5:

Andrés Del Negro
Federico Castiñeira

Profesor:

Walter José Aróztegui

3 de junio de 2025

Índice

1	Arquitectura de software	3
2	Funciones y comunicaciones	3
2.1	Modos de funcionamiento	3
2.2	Comunicaciones	3
2.3	Funciones de inicialización:	4
2.4	Funciones de operación	4
3	Diagrama de flujo	4
3.1	Estructura general	5
3.2	Seguidor	6
3.3	Configuración	7
3.4	Entregar mate, esperar mate y esperar cebador	8
4	Tareas para los ensayos	8
4.1	Medición de tiempos de ejecución	8
4.2	Ensayo de maniobra de giro	9
4.3	Ensayo de periféricos	9
4.4	Parámetros importantes	9
5	Esquemático	9
5.1	Consumos	9
5.2	Diagrama	11

1. Arquitectura de software

La arquitectura de software implementada corresponde a una Máquina de Estados Finitos (MEF)[4]. Las tareas se ejecutan a intervalos regulares utilizando un sistema de ticks. Para ello, se emplea la interrupción de un temporizador que incrementa una variable cada milisegundo. Esta variable se utiliza luego para sincronizar la ejecución de las distintas tareas, asignándoles distintos períodos de ejecución.

Esta estructura permite definir de forma clara los estados operativos del sistema y las transiciones entre ellos, lo que garantiza un comportamiento determinístico. Además, facilita el desarrollo, prueba y mantenimiento modular del software, favorece la incorporación de nuevas funcionalidades y permite una integración sencilla con la máquina de cebado (Cebático).

2. Funciones y comunicaciones

2.1. Modos de funcionamiento

- **Seguidor:** Las tareas incluidas en este modo son:
 - **Control:** Se efectúa la medición mediante el ADC, se aplica la acción de control utilizando un algoritmo PID, y luego se actualizan los registros del timer que generan las señales PWM para los motores.
 - **Ultrasonido:** Se activa el pulso de disparo (trigger) del sensor ultrasónico y se actualiza el valor de la variable de distancia.
 - **Comunicación:** Se verifica si hay información disponible desde el lector de códigos QR, o si se desea cambiar el modo de funcionamiento.
- **Configuración:** Este modo permite acceder a un menú donde se pueden modificar distintos parámetros del sistema:
 - **Cambiar parámetros del PID:** Permite modificar los valores de K_p y K_i .
 - **Cambiar parámetros de contraste:** Estos parámetros pueden ajustar colocando el sensor sobre la línea y luego sobre el fondo, para calibrar adecuadamente el contraste. Además, se debe seleccionar si el fondo será blanco o negro.
 - **Mostrar parámetros actuales:** Se muestran en pantalla los valores de los parámetros.
 - **Presencia del cebador:** Se configura si el sistema debe esperar la comunicación con la estación de cebado o la señal del botón en caso de que esta no se encuentre.
 - **Lista de personas presentes:** Se registra una lista con las personas presentes en la ronda. El móvil siempre debe iniciar desde el código QR de origen y retornar a él tras cada recorrido.
 - **Continuar recorrido:** Terminada la configuración el móvil pasa al modo seguidor.
- **Entregar mate:** En este modo, el móvil se orienta hacia el destino, detiene los motores y pasa al modo "Esperar mate".
- **Esperar mate:** El sistema permanece en espera hasta que se presione el botón de "mate listo". Luego, vuelve al modo "Seguidor" para regresar a la parada de origen.
- **Esperar cebador:** En este estado, el móvil permanece detenido hasta recibir una comunicación desde la estación de cebado que confirme que el mate está preparado, momento en el cual retoma su trayecto.

2.2. Comunicaciones

- **I2C:** El ATmega y el ESP32 se comunicarán mediante el protocolo I2C, donde el ATmega actuará como maestro y el ESP32 como esclavo. El ATmega solicitará al ESP32 información acerca de la detección de códigos QR; este último responderá negativamente si no se ha identificado ningún código, o proporcionará los datos correspondientes en caso afirmativo.
- **Wi-Fi:** El ESP32 se conectará al servidor a través de su interfaz de red utilizando el protocolo MQTT. Mediante esta conexión, el usuario podrá consultar el estado del móvil en tiempo real y realizar configuraciones de parámetros a través de una interfaz dedicada.

2.3. Funciones de inicialización:

- **init_ADC:** En esta función se configura el ADC para que utilice prescaler de 128 y tome como referencia a VCC.
- **init_ultrasonido:** Configura las interrupciones del Timer 1 para medir el pulso proveniente del sensor ultrasónico, relacionado con la distancia.
- **init_motores:** Inicializa el Timer 2 para generar las señales PWM que controlan los motores.
- **init_I2C:** Configura el sistema para establecer la comunicación I2C entre el ATmega y el ESP32.

2.4. Funciones de operación

- **ADC:** Se realiza las 5 mediciones de los sensores infrarrojos, la compara con el valor del contraste y determina la posición de la línea.
- **PID:** A partir de la posición de la línea, se calculan los términos proporcional e integral del controlador. El valor resultante se utiliza para modificar la velocidad de los motores.
- **Velocidad:** Se actualizan los registros del Timer 2 para ajustar la velocidad de los motores según la salida del PID.
- **I2C:** Se solicita información al ESP32 relacionada con la detección de códigos QR o con el modo de configuración.

3. Diagrama de flujo

A continuación, se presenta el diagrama de flujo correspondiente a la Máquina de Estados Finitos (MEF) implementada. Con el objetivo de facilitar la comprensión y visualización del comportamiento del sistema, se ha optado por desglosar el flujo de ejecución en diagramas individuales, uno por cada estado operativo definido.

El programa puede transitar entre los siguientes cinco estados principales:

1. **Seguidor:** Encargado del seguimiento de líneas mediante sensores.
2. **Configuración:** Permite ajustar parámetros del sistema, como el PID, el contraste o la configuración de usuarios.
3. **Entregar Mate:** El móvil se detiene frente al destino para entregar el mate.
4. **Esperar Mate:** El sistema permanece en espera hasta que se confirme que el mate fue preparado.
5. **Esperar Cebador:** El sistema aguarda una señal desde la máquina cebadora (Cebático) antes de continuar.

3.1. Estructura general

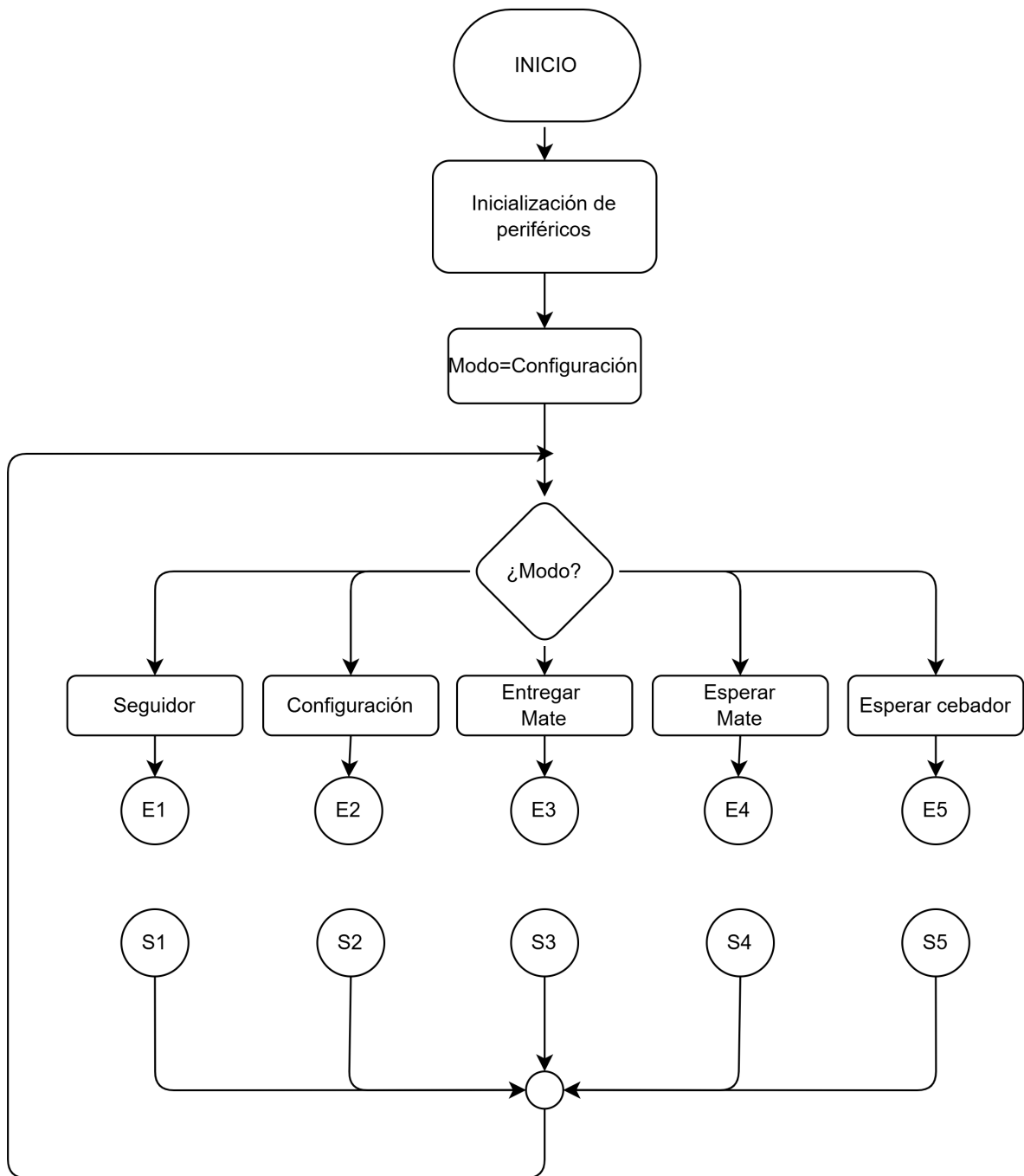


Figura 1: Diagrama de flujo de la estructura general del programa.

3.2. Seguidor

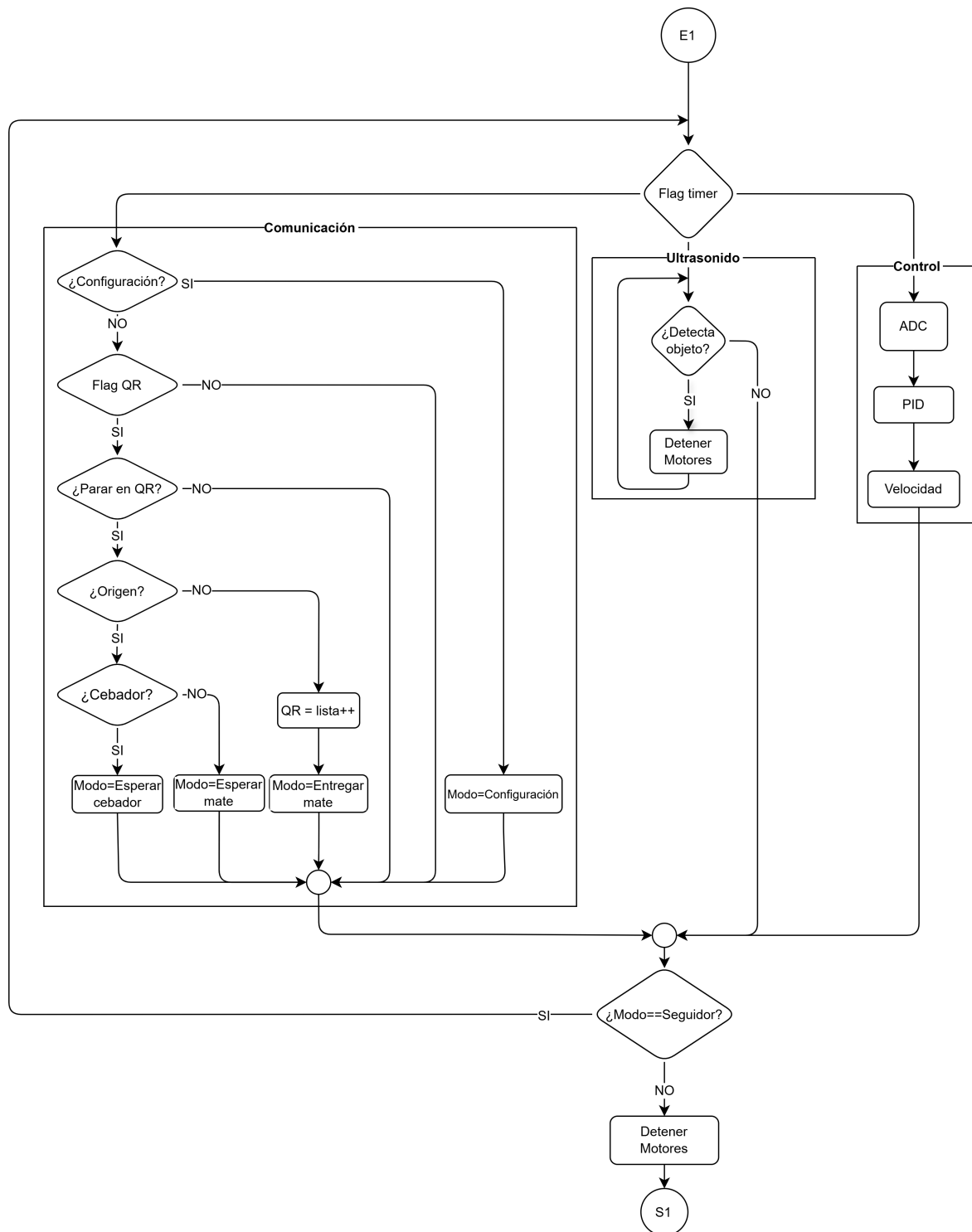


Figura 2: Diagrama de flujo del modo seguidor.

3.3. Configuración

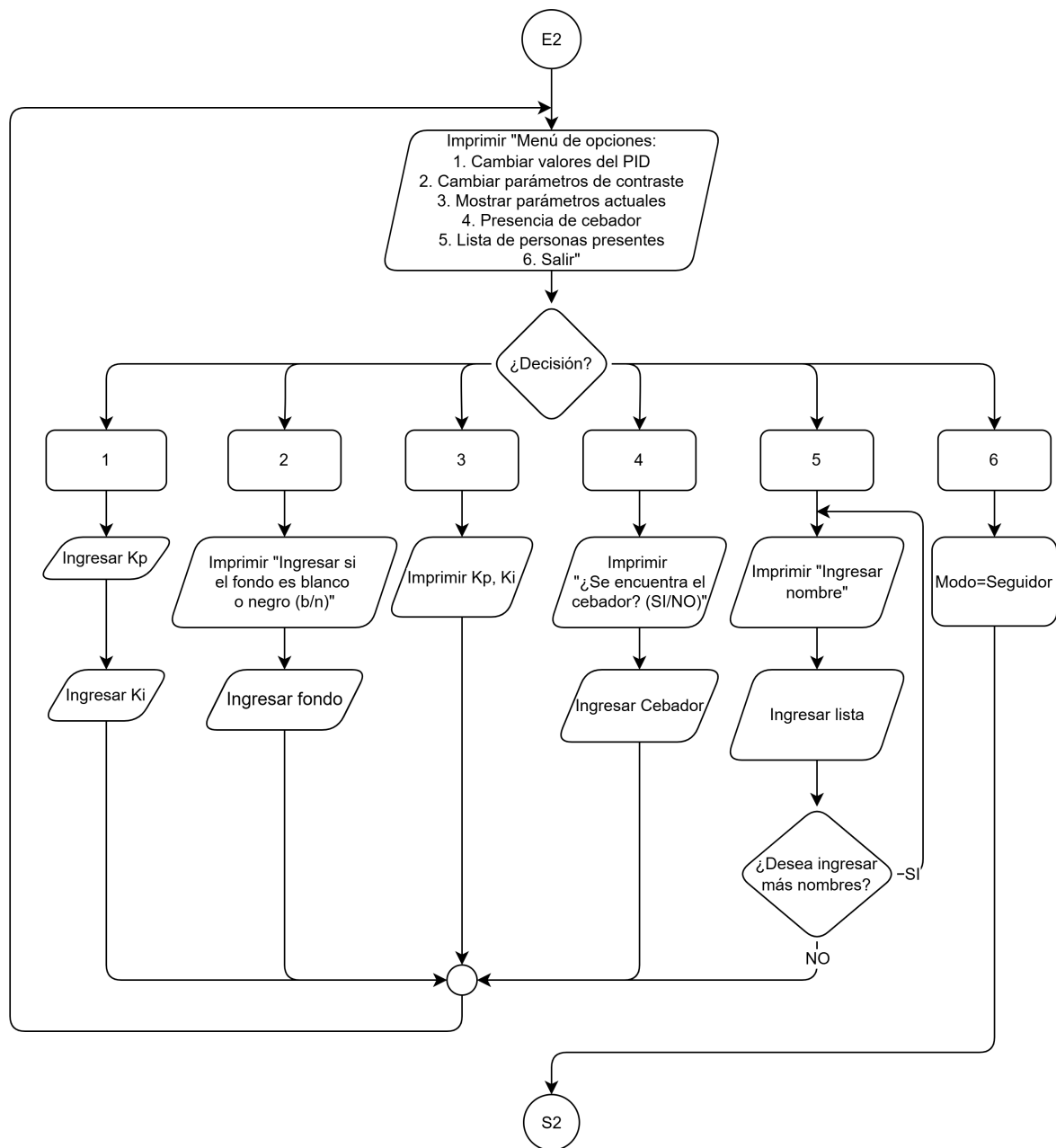


Figura 3: Diagrama de flujo del modo configuración.

3.4. Entregar mate, esperar mate y esperar cebador

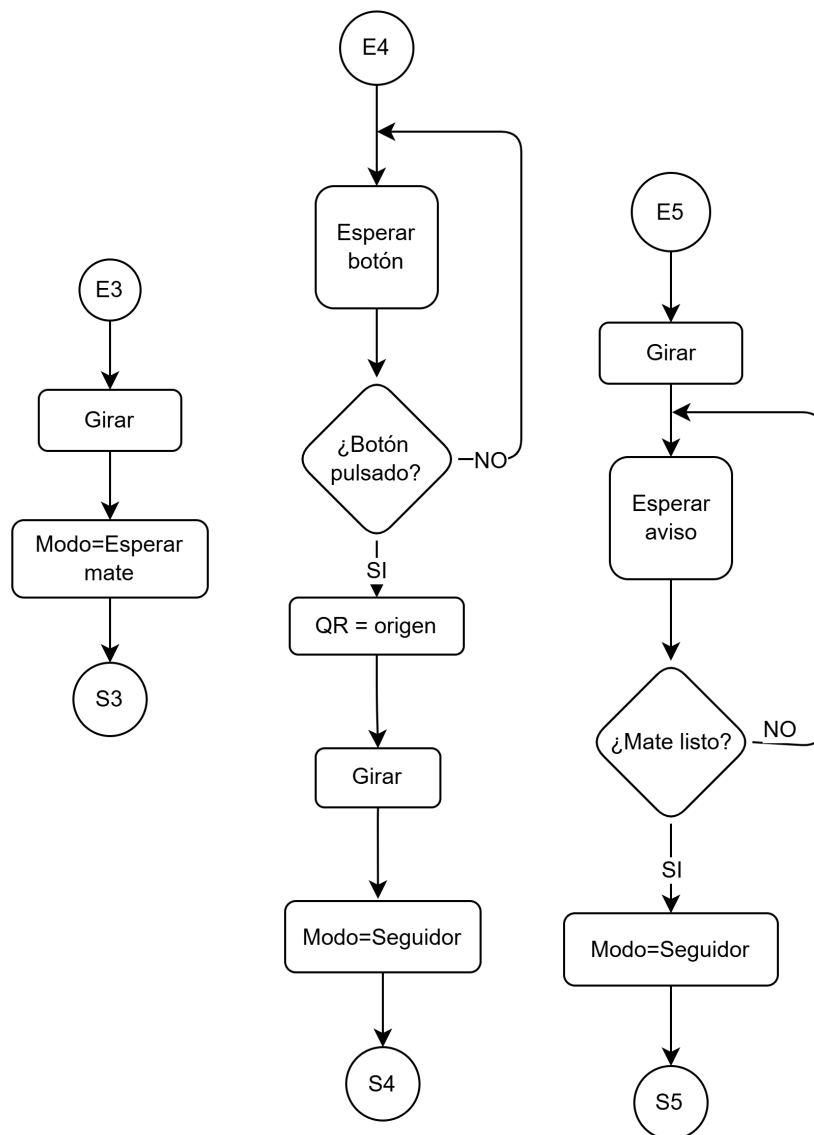


Figura 4: Diagrama de flujo de los modos entregar mate, esperar mate y esperar cebador.

4. Tareas para los ensayos

Para validar el correcto funcionamiento del sistema y optimizar su rendimiento, se realizarán una serie de ensayos experimentales. El objetivo principal de estos ensayos es determinar los tiempos de ejecución de las tareas críticas, además de caracterizar el comportamiento dinámico del móvil durante acciones específicas. Las tareas a realizar se detallan a continuación:

4.1. Medición de tiempos de ejecución

Se medirán los tiempos requeridos para la ejecución de las siguientes operaciones críticas:

- Medir el tiempo de conversión analógica-digital (ADC) para cada sensor infrarrojo.
- Medir el tiempo requerido para ejecutar el algoritmo PID y la actualización de los registros PWM.

- Evaluar el tiempo necesario para establecer y completar una transacción I2C entre el ATmega (maestro) y el ESP32 (esclavo), implementando para ello una rutina de prueba sencilla que permita el intercambio de datos básicos entre ambos dispositivos.

Estas mediciones permitirán ajustar los períodos de ejecución de las tareas en función de sus tiempos reales, garantizando la correcta sincronización y desempeño del sistema. A partir de estos ensayos, se determinará también el tiempo de muestreo óptimo para el algoritmo de control, asegurando una ejecución eficiente y estable.

4.2. Ensayo de maniobra de giro

Se debe implementar una función de giro para el móvil que permita realizar rotaciones precisas. En particular, se requiere:

- Programar un giro de 90 grados: para ello es necesario medir el tiempo necesario para que los motores completen dicha rotación, evaluando además la precisión y repetibilidad del movimiento.

Esta prueba permitirá establecer un valor de referencia temporal para movimientos angulares, útil para la planificación de las maniobras del móvil.

4.3. Ensayo de periféricos

Se realizarán pruebas de funcionamiento para cada periférico y se desarrollarán las respectivas librerías. Las pruebas incluyen:

- Motores: Configuración y prueba de un timer del ATmega en modo PWM para control de velocidad.
- Sensor ultrasónico: Uso de un timer para medir el tiempo del pulso que devuelve el sensor para la determinación de la distancia.
- Comunicación I2C: Verificación del correcto funcionamiento de las librerías de comunicación entre el ATmega y el ESP32.

Estos ensayos garantizarán la integración correcta y estable de los dispositivos en el sistema.

4.4. Parámetros importantes

Las constantes del algoritmo de control serán ajustadas mediante un proceso de prueba y error, evaluando el desempeño del sistema ante distintos escenarios y condiciones de operación.

Por otra parte, los valores de referencia utilizados para el seguimiento de línea se establecerán a partir de mediciones repetidas sobre una superficie representativa del entorno real de funcionamiento, con el objetivo de garantizar una detección confiable y consistente.

5. Esquemático

Es necesario incorporar un módulo de conversión lógica entre el ATmega y el ESP32, ya que este último opera a 3.3V, mientras que el ATmega funciona a 5V

Las baterías serán la fuente de alimentación de la electrónica.

5.1. Consumos

- Motores:

Característica	Valor aproximado
Voltaje típico	3 V – 6 V
Corriente sin carga	80 – 100 mA
Corriente bajo carga ligera	100 – 200 mA

Corriente en arranque/bloqueo	300 – 500 mA
-------------------------------	--------------

Cuadro 1: Consumo típico de los motores de corriente continua.

- Módulo infrarrojo: Trabaja durante un tiempo muy reducido por lo que no se considera de un consumo relevante.
- ATmega328p[1]:

Modo de operación	Consumo típico (mA)
Activo (código ejecutándose)	9 – 12
Idle (CPU detenida, periféricos activos)	2 – 3
ADC Noise Reduction Mode	0.5 – 1
Power-down (modo suspensión profunda)	0.1 – 0.2
Power-save (modo suspensión con RTC activo)	0.001 – 0.005
Standby	0.0005 – 0.002

Cuadro 2: Consumo típico del ATmega328P a 16 MHz y 5 V

- Ultrasonido[3]:

Modo de operación	Consumo típico (mA)
Sensor inactivo (en espera)	2
Sensor activo (medición puntual)	15
Sensor activo (mediciones continuas)	15 – 20

Cuadro 3: Consumo típico de corriente del sensor ultrasónico HC-SR04

- ESP32[5]:

Modo de operación	Consumo típico (mA)
Encendido, cámara y WiFi activos (uso máximo)	300 – 500
WiFi activo, cámara inactiva	160 – 250
Cámara activa, WiFi desactivado	100 – 180
Solo procesador activo (sin periféricos)	80 – 100
Modo de espera (light sleep)	6 – 20
Modo deep sleep	0.2 – 1.5
Apagado (power off total)	< 0.1

Cuadro 4: Consumo típico de corriente del módulo ESP32-CAM en diferentes modos

Características de las baterías:

Parámetro	Valor específico
Tipo	Ion de litio (Li-ion)
Voltaje nominal	3.7 V
Voltaje de carga completa	4.2 V
Voltaje de corte (descarga)	3.0 V
Capacidad	2500 mAh
Corriente de descarga recomendada	1250 mA
Corriente de descarga máxima	2500 mA
Corriente de carga recomendada	1250 mA
Corriente de carga máxima	2500 mA
Protección interna	No (requiere protección externa si es sin PCB)
Dimensiones	18 mm × 65 mm
Peso	45 g
Ciclos de vida útil	≥ 300 ciclos (al 100 % de descarga)

Cuadro 5: Características eléctricas de la batería Motoma 18650 de 2500 mAh

En conclusión, podemos decir que la batería va a cumplir de manera correcta con la corriente demandada por los periféricos. Esta va a ser de alrededor de 1A.

5.2. Diagrama

En el diagrama esquemático[2] completo del proyecto, que se puede ver en la Figura 5, se muestra la conexión de cada uno de los periféricos, incluyendo la entrada correspondiente para la alimentación y un puerto para la programación del ESP32CAM.

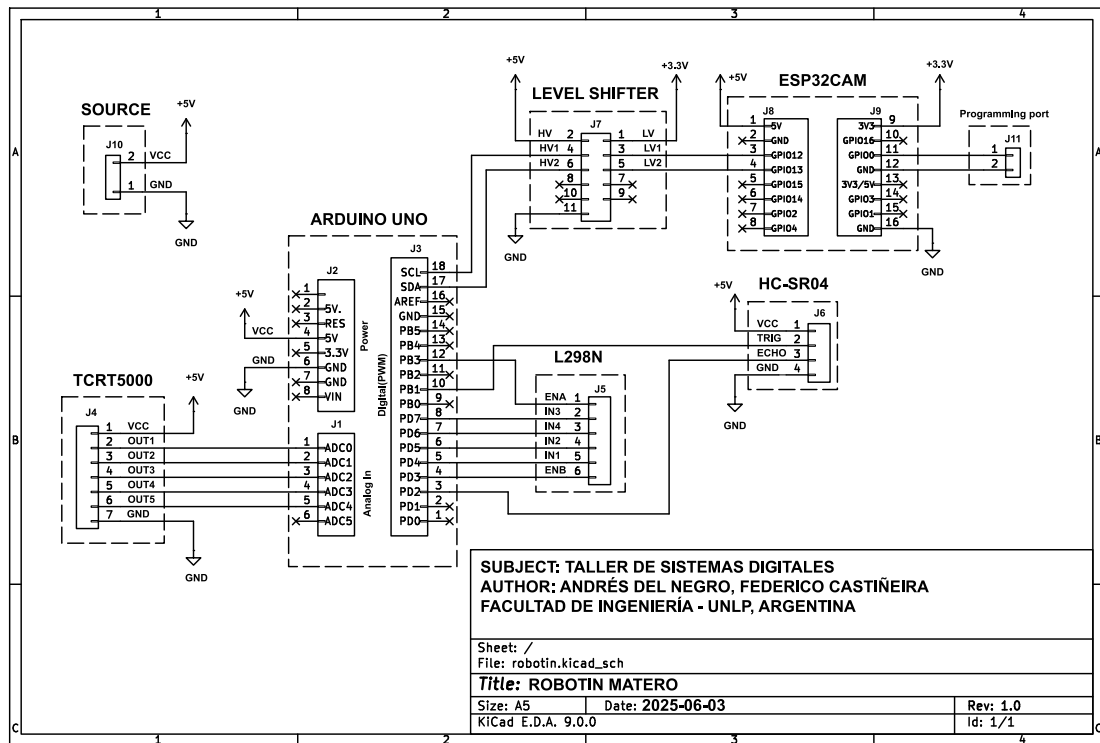


Figura 5: Diagrama esquemático.

Referencias

- [1] Atmel. *8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash DATASHEET*. Accedido: 2025-06-03. 2015. URL: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.
- [2] Diego Brengi. *Procedimientos para realizar un circuito esquemático*. Accedido: 2025-06-03. 2017. URL: https://www.asignaturas.ing.unlp.edu.ar/pluginfile.php/375294/mod_resource/content/1/Brengi_Procedimientos_Dise%C3%B1o_Esquemático.pdf.
- [3] SparkFun Electronics. *HC-SR04 Ultrasonic Sensor Datasheet*. Accedido: 2025-04-26. 2025. URL: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>.
- [4] Cátedra Taller de Sistemas Digitales. *Esquemas de Software*. Accedido: 2025-06-03. 2025. URL: https://www.asignaturas.ing.unlp.edu.ar/pluginfile.php/375475/mod_resource/content/1/Esquemas%20de%20Software.pdf.
- [5] Espressif Systems. *ESP32 Datasheet*. Accedido: 2025-06-03. 2016. URL: https://cdn.sparkfun.com/datasheets/IoT/esp32_datasheet_en.pdf.